

✓ Explainable Customer Churn Prediction Using XAI

Project Objective

The goal of this project is to build a machine learning model that predicts customer churn in a telecommunications company and explains the model's decisions using Explainable Artificial Intelligence methods.

The project focuses on two main questions:

1. Which customers are likely to leave the company?
2. Why are these customers predicted as high-risk?

This is important from a business perspective because churn prediction can help the company design targeted retention actions, reduce customer loss, and improve profitability.

✓ Business Interpretation

The model does not only predict churn. It also explains the main reasons behind churn risk.

The XAI results can help the company identify high-risk customer groups and create retention strategies.

Possible business actions include:

1. Target customers with month-to-month contracts.
2. Offer incentives for switching to one-year or two-year contracts.
3. Provide special support to customers with high monthly charges.
4. Promote automatic payment methods.
5. Improve technical support and online security services.

These actions are directly connected to the factors identified by the XAI methods.

```
import sys
from pathlib import Path

PROJECT_ROOT = Path.cwd()

SRC_DIR = PROJECT_ROOT / "src"

if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))

print("Project root:", PROJECT_ROOT)
print("Source folder:", SRC_DIR)
```

```
Project root: C:\Users\Yuceltan Ebiri\PycharmProjects\Telco-Customer-Churn
Source folder: C:\Users\Yuceltan Ebiri\PycharmProjects\Telco-Customer-Churn\src
```

✓ Dataset Description

The dataset used in this project is the Telco Customer Churn dataset from Kaggle.

Each row represents one customer. The target variable is `Churn`, which indicates whether the customer left the company.

The dataset contains business-readable features such as:

- Contract type
- Tenure
- Monthly charges
- Total charges
- Internet service
- Online security
- Tech support
- Payment method
- Paperless billing

These features are suitable for Explainable AI because they can be understood by non-technical business stakeholders.

```
from src.config import DATA_PATH
from src.data_preprocessing import load_data

df = load_data(DATA_PATH)

print("Dataset shape:", df.shape)
df.head()
```

Dataset shape: (7043, 21)

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No

5 rows × 21 columns

```
df = load_data(DATA_PATH)

print("Dataset shape:", df.shape)
df.head()
```

Dataset shape: (7043, 21)

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No

5 rows × 21 columns

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customerID            7043 non-null   str
1   gender                 7043 non-null   str
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   str
4   Dependents             7043 non-null   str
5   tenure                 7043 non-null   int64
6   PhoneService           7043 non-null   str
7   MultipleLines          7043 non-null   str
8   InternetService        7043 non-null   str
9   OnlineSecurity         7043 non-null   str
10  OnlineBackup           7043 non-null   str
11  DeviceProtection       7043 non-null   str
12  TechSupport            7043 non-null   str
13  StreamingTV            7043 non-null   str
14  StreamingMovies        7043 non-null   str
15  Contract               7043 non-null   str
16  PaperlessBilling       7043 non-null   str
```

```
17 PaymentMethod      7043 non-null    str
18 MonthlyCharges      7043 non-null    float64
19 TotalCharges        7043 non-null    str
20 Churn               7043 non-null    str
dtypes: float64(1), int64(2), str(18)
memory usage: 1.1 MB
```

```
df.describe(include="all")
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	Online
count	7043	7043	7043.000000	7043	7043	7043.000000	7043	7043	7043	
unique	7043	2	NaN	2	2	NaN	2	3	3	
top	7590-VHVEG	Male	NaN	No	No	NaN	Yes	No	Fiber optic	
freq	1	3555	NaN	3641	4933	NaN	6361	3390	3096	
mean	NaN	NaN	0.162147	NaN	NaN	32.371149	NaN	NaN	NaN	
std	NaN	NaN	0.368612	NaN	NaN	24.559481	NaN	NaN	NaN	
min	NaN	NaN	0.000000	NaN	NaN	0.000000	NaN	NaN	NaN	
25%	NaN	NaN	0.000000	NaN	NaN	9.000000	NaN	NaN	NaN	
50%	NaN	NaN	0.000000	NaN	NaN	29.000000	NaN	NaN	NaN	
75%	NaN	NaN	0.000000	NaN	NaN	55.000000	NaN	NaN	NaN	
max	NaN	NaN	1.000000	NaN	NaN	72.000000	NaN	NaN	NaN	

11 rows × 11 columns

▼ Data Cleaning

The dataset requires several cleaning steps before model training.

The following actions are performed:

- 1. `customerID` is removed because it is only an identifier.
- 2. `TotalCharges` is converted from text to numeric format.
- 3. Missing values in `TotalCharges` are removed.
- 4. The target variable `Churn` is converted into binary format:
 - Yes = 1
 - No = 0

```
from src.data_preprocessing import clean_data

df_clean = clean_data(df)

print("Original shape:", df.shape)
print("Cleaned shape:", df_clean.shape)

df_clean.head()
```

Original shape: (7043, 21)
Cleaned shape: (7032, 20)

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup
0	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes
1	Male	0	No	No	34	Yes	No	DSL	Yes	No
2	Male	0	No	No	2	Yes	No	DSL	Yes	Yes
3	Male	0	No	No	45	No	No phone service	DSL	Yes	No
4	Female	0	No	No	2	Yes	No	Fiber optic	No	No

```
df_clean.isna().sum()
```

```
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
Churn           0
dtype: int64
```

✓ Exploratory Data Analysis

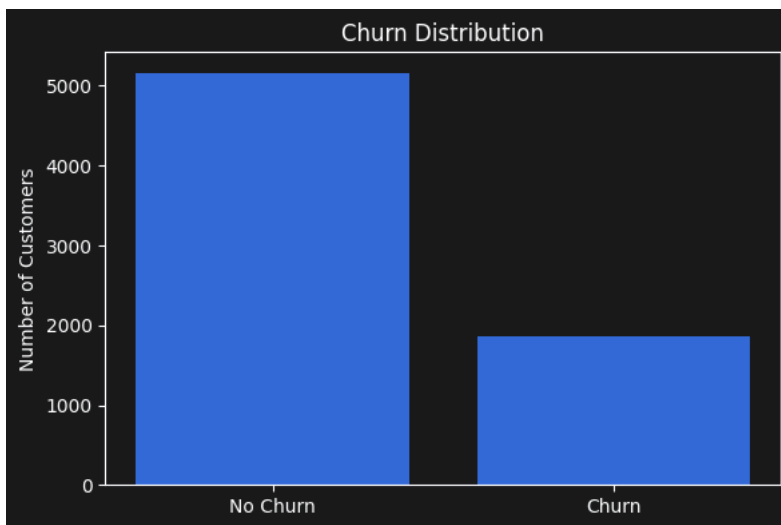
The aim of this section is to understand the main patterns in customer churn before building the model.

```
from matplotlib import pyplot as plt

churn_counts = df_clean["Churn"].value_counts()

plt.figure(figsize=(6, 4))
plt.bar(["No Churn", "Churn"], churn_counts.values)
plt.title("Churn Distribution")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()

churn_counts
```



```
Churn
0    5163
1    1869
Name: count, dtype: int64
```

```
churn_rate = df_clean["Churn"].mean()
print(f"Overall churn rate: {churn_rate:.2%}")
```

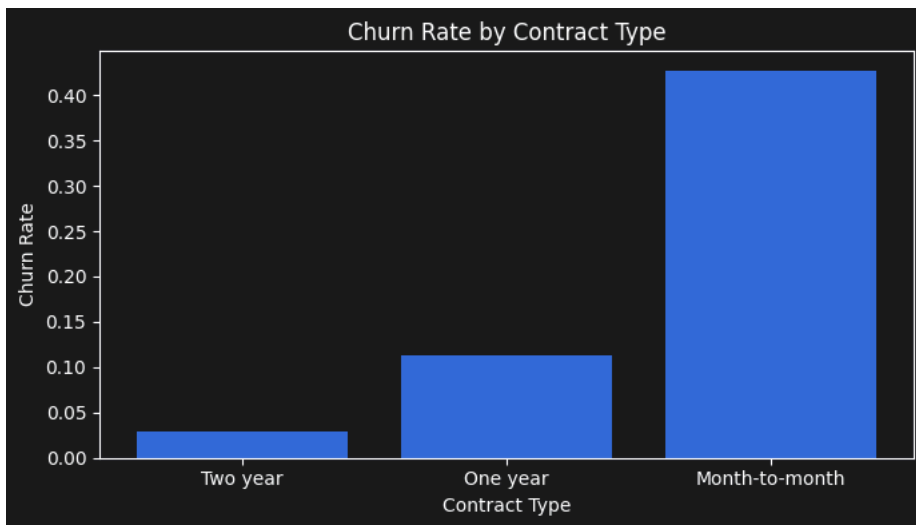
```
Overall churn rate: 26.58%
```

```
contract_churn = df_clean.groupby("Contract")["Churn"].mean().sort_values()

plt.figure(figsize=(7, 4))
plt.bar(contract_churn.index, contract_churn.values)
```

```
plt.title("Churn Rate by Contract Type")
plt.ylabel("Churn Rate")
plt.xlabel("Contract Type")
plt.tight_layout()
plt.show()
```

contract_churn

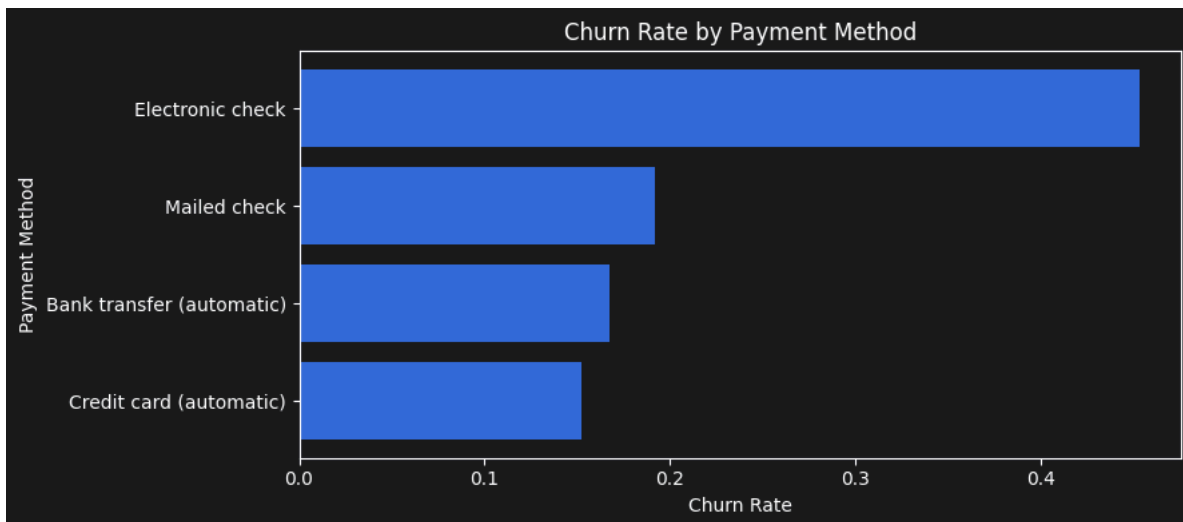


```
Contract
Two year      0.028487
One year      0.112772
Month-to-month 0.427097
Name: Churn, dtype: float64
```

```
payment_churn = df_clean.groupby("PaymentMethod")["Churn"].mean().sort_values()
```

```
plt.figure(figsize=(9, 4))
plt.barh(payment_churn.index, payment_churn.values)
plt.title("Churn Rate by Payment Method")
plt.xlabel("Churn Rate")
plt.ylabel("Payment Method")
plt.tight_layout()
plt.show()
```

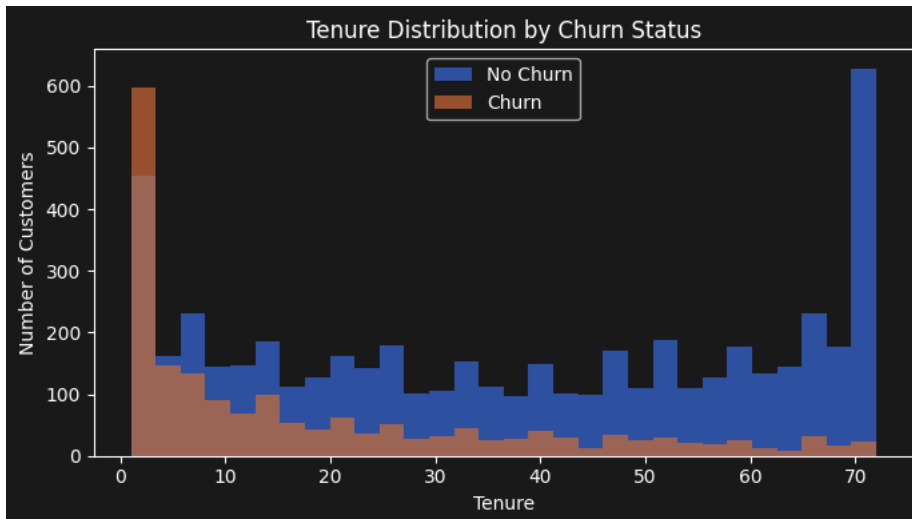
payment_churn



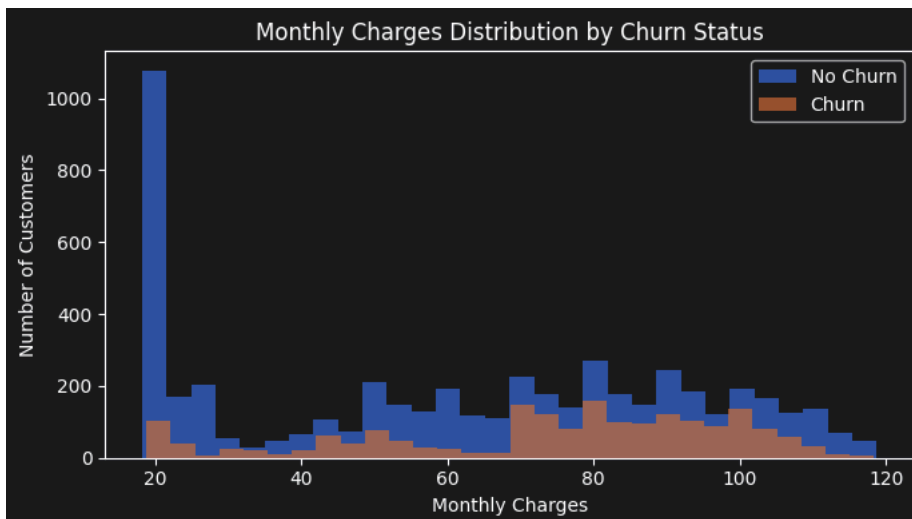
```
PaymentMethod
Credit card (automatic)    0.152531
Bank transfer (automatic)  0.167315
Mailed check                0.192020
Electronic check           0.42854
Name: Churn, dtype: float64
```

```
plt.figure(figsize=(7, 4))
plt.hist(df_clean[df_clean["Churn"] == 0]["tenure"], bins=30, alpha=0.7, label="No Churn")
plt.hist(df_clean[df_clean["Churn"] == 1]["tenure"], bins=30, alpha=0.7, label="Churn")
```

```
plt.title("Tenure Distribution by Churn Status")
plt.xlabel("Tenure")
plt.ylabel("Number of Customers")
plt.legend()
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(7, 4))
plt.hist(df_clean[df_clean["Churn"] == 0]["MonthlyCharges"], bins=30, alpha=0.7, label="No Churn")
plt.hist(df_clean[df_clean["Churn"] == 1]["MonthlyCharges"], bins=30, alpha=0.7, label="Churn")
plt.title("Monthly Charges Distribution by Churn Status")
plt.xlabel("Monthly Charges")
plt.ylabel("Number of Customers")
plt.legend()
plt.tight_layout()
plt.show()
```



✓ Train-Test Split and Preprocessing

The target variable is separated from the features.

The dataset is split into training and test sets using stratification. This keeps the churn ratio similar in both sets.

Numerical features are standardized, while categorical features are transformed using one-hot encoding.

```
from src.data_preprocessing import split_features_target, create_train_test_split

X, y = split_features_target(df_clean)

X_train, X_test, y_train, y_test = create_train_test_split(X, y)
```

```
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train churn rate:", y_train.mean())
print("y_test churn rate:", y_test.mean())
```

```
X_train shape: (5625, 19)
X_test shape: (1407, 19)
y_train churn rate: 0.2657777777777778
y_test churn rate: 0.2658137882018479
```

```
from src.data_preprocessing import build_preprocessor

preprocessor, numerical_features, categorical_features = build_preprocessor(X_train)

print("Numerical features:")
print(numerical_features)

print("\nCategorical features:")
print(categorical_features)
```

```
Numerical features:
['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCharges']

Categorical features:
['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection']
```

Model Training

Two models are trained:

- 1. Logistic Regression This model is used as a baseline because it is simple and interpretable.
- 2. Random Forest This model is used as the black-box model. The main XAI methods are applied to this model.

```
from src.train_model import train_models

trained_models = train_models(preprocessor, X_train, y_train)
```

```
Training model: Logistic Regression
Training model: Random Forest
```

Model Evaluation

The models are evaluated using several metrics:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

In churn prediction, recall is especially important because the company wants to identify as many customers at risk of churn as possible.

```
from src.evaluate_model import evaluate_models

metrics_df = evaluate_models(trained_models, X_test, y_test)
metrics_df
```

```
Evaluating model: Logistic Regression
Evaluating model: Random Forest
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	Logistic Regression	0.725657	0.490132	0.796791	0.606925	0.835134
1	Random Forest	0.766880	0.545455	0.737968	0.627273	0.831904

```
best_model_name = metrics_df.sort_values(by="ROC AUC", ascending=False).iloc[0]["Model"]
best_model_name
```

```
'Logistic Regression'
```

✓ XAI Method 1: Permutation Feature Importance

Permutation feature importance explains how important each original feature is for the model.

The logic is simple:

- A feature is randomly shuffled.
- The model performance is measured again.
- If model performance decreases strongly, the feature is important.

This method is useful for non-technical stakeholders because it provides a direct ranking of business factors influencing churn prediction.

```
from src.explain_xai import save_permutation_importance

xai_model = trained_models["Random Forest"]

permutation_importance_df = save_permutation_importance(
    xai_model,
    X_test,
    y_test
)

permutation_importance_df.head(15)
```

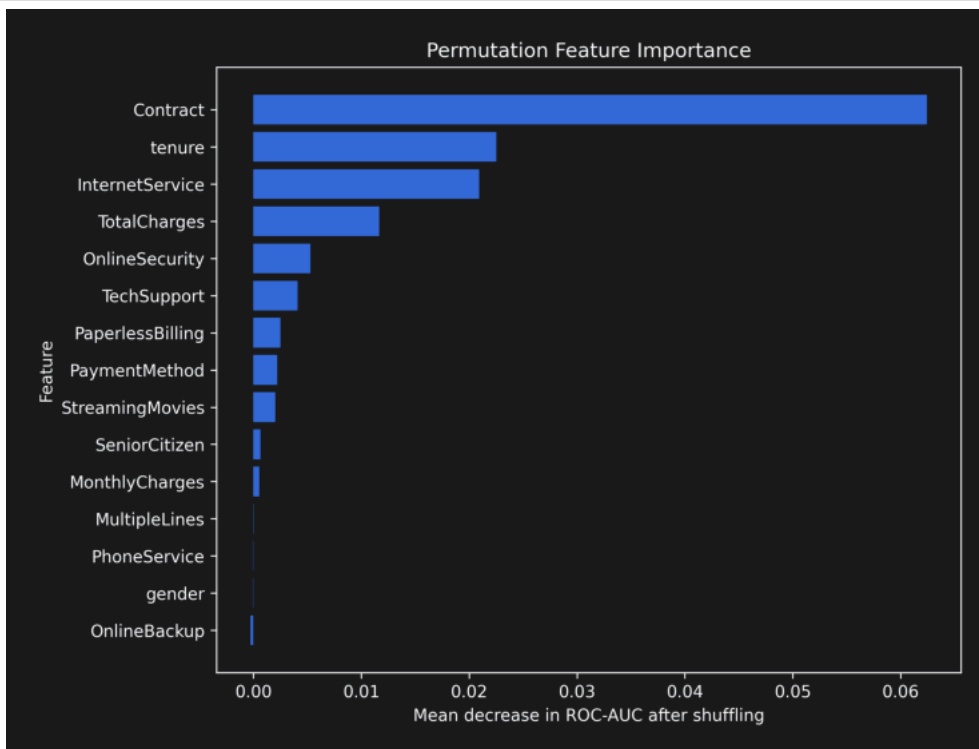
Calculating permutation importance...

	feature	importance_mean	importance_std
14	Contract	0.062463	0.009187
4	tenure	0.022539	0.004609
7	InternetService	0.020926	0.004843
18	TotalCharges	0.011665	0.003138
8	OnlineSecurity	0.005296	0.002947
11	TechSupport	0.004105	0.002014
15	PaperlessBilling	0.002510	0.002274
16	PaymentMethod	0.002198	0.001900
13	StreamingMovies	0.002036	0.001533
1	SeniorCitizen	0.000643	0.000605
17	MonthlyCharges	0.000540	0.002438
6	MultipleLines	0.000029	0.001756
5	PhoneService	-0.000045	0.000269
0	gender	-0.000071	0.000681
9	OnlineBackup	-0.000281	0.001265

```
from src.config import FIGURES_DIR

img = plt.imread(FIGURES_DIR / "permutation_feature_importance.png")

plt.figure(figsize=(10, 7))
plt.imshow(img)
plt.axis("off")
plt.show()
```

~ XAI Method 2: SHAP Global Explanation

SHAP explains how each feature contributes to model predictions.

A global SHAP explanation shows which features are most influential across many customers.

This allows the company to understand the general drivers.

```
from src.explain_xai import save_shap_global_explanations

shap_importance_df, explainer, X_processed_df, shap_values_churn = save_shap_global_explanations(
    xai_model,
    X_test
)

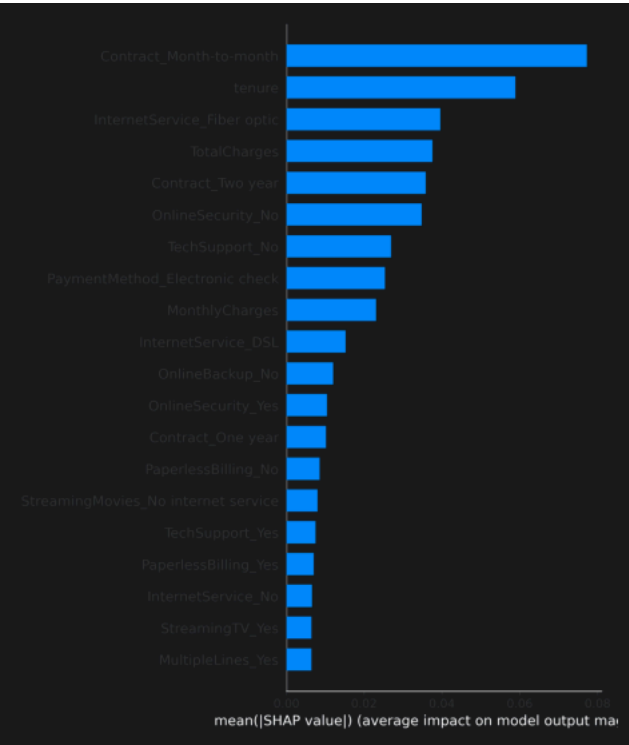
shap_importance_df.head(20)
```

Creating SHAP global explanations...

	feature	mean_absolute_shap
36	Contract_Month-to-month	0.077216
1	tenure	0.058857
16	InternetService_Fiber optic	0.039570
3	TotalCharges	0.037535
38	Contract_Two year	0.035840
18	OnlineSecurity_No	0.034810
27	TechSupport_No	0.026928
43	PaymentMethod_Electronic check	0.025366
2	MonthlyCharges	0.023047
15	InternetService_DSL	0.015221
21	OnlineBackup_No	0.012054
20	OnlineSecurity_Yes	0.010484
37	Contract_One year	0.010197
39	PaperlessBilling_No	0.008583
34	StreamingMovies_No internet service	0.008033
29	TechSupport_Yes	0.007511
40	PaperlessBilling_Yes	0.007063
17	InternetService_No	0.006557
32	StreamingTV_Yes	0.006483
14	MultipleLines_Yes	0.006459

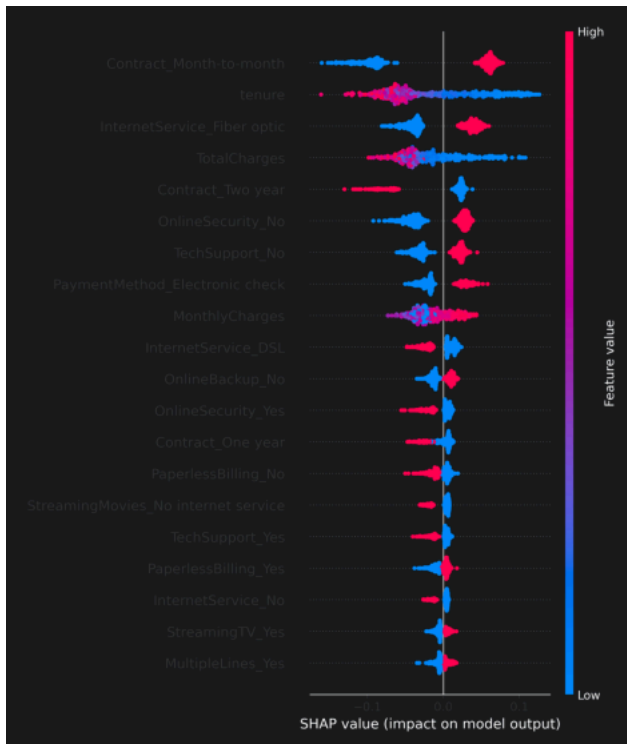
```
img = plt.imread(FIGURES_DIR / "shap_summary_bar.png")

plt.figure(figsize=(10, 7))
plt.imshow(img)
plt.axis("off")
plt.show()
```



```
img = plt.imread(FIGURES_DIR / "shap_summary_beeswarm.png")

plt.figure(figsize=(10, 7))
plt.imshow(img)
plt.axis("off")
plt.show()
```



✓ XAI Method 3: SHAP Local Explanation

A local SHAP explanation explains one individual customer prediction.

This is useful because the company can understand why a specific customer is classified as high-risk.

For example, the model may predict high churn risk because a customer has:

- Month-to-month contract
- Short tenure
- High monthly charges
- No technical support
- Electronic check payment

```
from src.explain_xai import save_shap_local_explanation
```

```
highest_risk_customer = save_shap_local_explanation(
    xai_model,
    X_test
)
```

```
highest_risk_customer
```

Creating SHAP local explanation...

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup
3727	Female	0	No	No	3	Yes	Yes	Fiber optic	No	

```
waterfall_path = FIGURES_DIR / "shap_local_waterfall_high_risk_customer.png"
bar_path = FIGURES_DIR / "shap_local_bar_high_risk_customer.png"
```

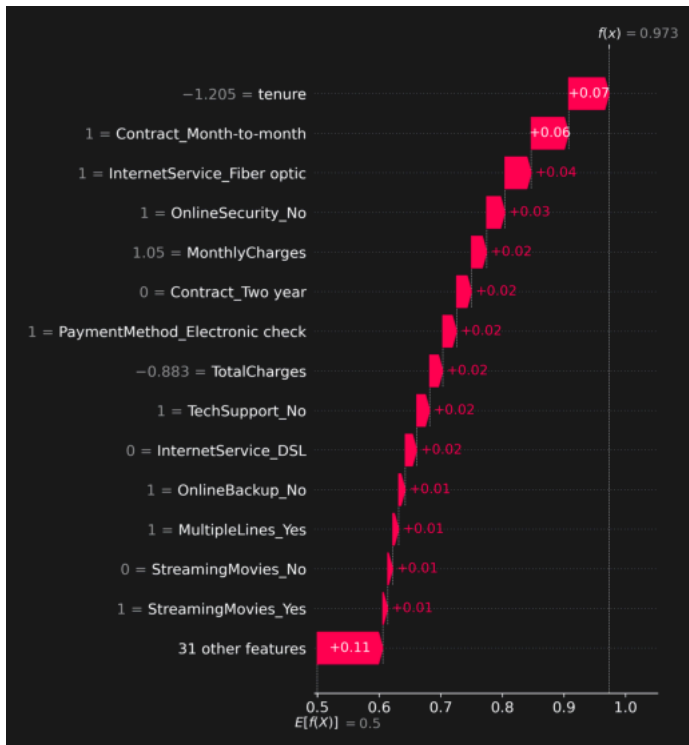
```
if waterfall_path.exists():
    img = plt.imread(waterfall_path)
```

```

else:
    img = plt.imread(bar_path)

plt.figure(figsize=(10, 7))
plt.imshow(img)
plt.axis("off")
plt.show()

```



Optional XAI Extension: What-if Analysis

A simple what-if analysis is used to estimate how the predicted churn probability changes under different possible business actions.

For example:

- What if the customer changed from a month-to-month contract to a two-year contract?
- What if the customer used automatic payment?
- What if the customer added tech support?

This is useful because it translates model explanations into possible retention actions.

```

from src.explain_xai import save_counterfactual_analysis

counterfactual_df = save_counterfactual_analysis(
    xai_model,
    X_test
)

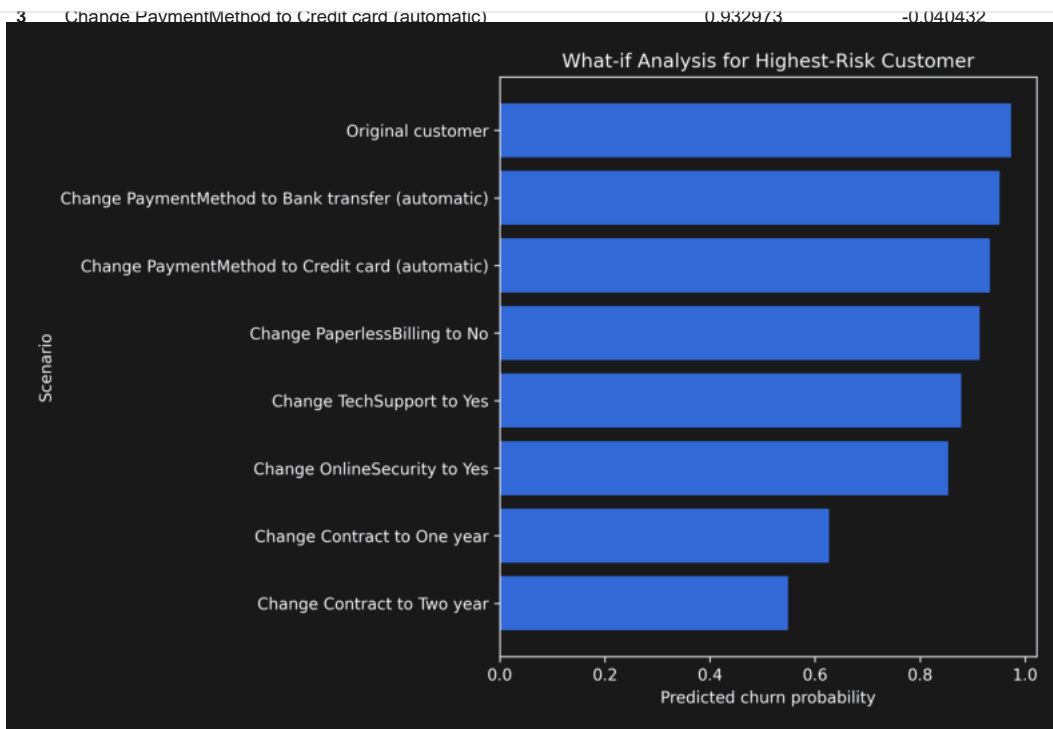
counterfactual_df

```

Creating counterfactual what-if analysis...

```
img = plt.imread(FIGURES_DIR / "counterfactual_what_if_analysis.png")

plt.figure(figsize=(10, 7))
plt.imshow(img)
plt.axis("off")
plt.show()
```



Business Interpretation

The model does not only predict churn. It also explains the main reasons behind churn risk.

The XAI results can help the company identify high-risk customer groups and create retention strategies.

Possible business actions include:

1. Target customers with month-to-month contracts.
2. Offer incentives for switching to one-year or two-year contracts.
3. Provide special support to customers with high monthly charges.